Providing Certified Triggers in Trigger Set-Based Watermarking for Ownership Validation against Image Transformation

Jingyang Li
Shanghai Jiao Tong University
Shanghai, China
lijjjjj@sjtu.edu.cn

Minyu Chen
Shanghai Jiao Tong University
Shanghai, China
minkow@sjtu.edu.cn

Guoqiang Li*
Shanghai Jiao Tong University
Shanghai, China
li.g@sjtu.edu.cn

Abstract

Deep Neural Network (DNN) watermarking techniques are extensively utilized to safeguard the intellectual property of neural networks. Numerous watermarking schemes based on trigger sets have been proposed, claiming to possess robustness. However, the claimed robustness is typically validated only against watermark removal attacks that involve manipulation of network weights. These schemes lack systematic evaluation and formal guarantees for perturbed input-based attacks. Consequently, the uncertainty surrounding the robustness of watermarking schemes hinders their practical deployment and trustworthiness. In this research paper, we contend that existing watermarking algorithms may not fulfill their objective of claiming ownership. The current defense techniques for watermark removal primarily focus on attacks that modify the network, such as parameter and structure changes. However, they disregard the fact that adversaries can undermine ownership verification by simply distorting the watermarked image. To address this oversight, we formally define the robustness of neural network watermarks against three types of attacks: adversarial attacks, positional attacks, and alpha channel attacks. Furthermore, we comprehensively evaluate four representative watermarking schemes to provide formal guarantees regarding the three types of robustness mentioned above. Our evaluation is conducted using the CIFAR-10 and MNIST image classification datasets. The results of our study demonstrate that none of the surveyed watermarking schemes exhibit practical robustness. This highlights a lack of consideration for simple input-based attacks during the training process of watermarking schemes. Therefore, it is imperative to provide watermarking schemes with formal guarantees before their implementation in practical settings.

CCS Concepts

• **Software and its engineering → Formal software verification**.

Keywords

Deep Neural Network Watermarking, Neural Network Verification

ACM Reference Format:
Jingyang Li, Minyu Chen, and Guoqiang Li. 2025. Providing Certified Triggers in Trigger Set-Based Watermarking for Ownership Validation against Image Transformation. In *Proceedings of the 2025 Workshop on Recent Advances in Resilient and Trustworthy MAchine learning-driveN systems (ARTMAN '25), October 13–17, 2025, Taipei, Taiwan.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3733821.3763024

1 Introduction

Watermarking is a critical technology for protecting intellectual property rights in the realm of computer vision. It involves embedding distinctive information, such as university logos or ownership descriptions, into input image data to assert copyright ownership over neural network outputs. Recent advances in watermarking techniques have largely relied on trigger set-based methods [1, 5, 7, 9, 24], which offer explicit identifiers to deter unauthorized use and establish ownership unequivocally.

One primary problem with trigger set-based watermarking techniques is that such watermarks might be easy to erase, and can lead to unwilling consequences. To overcome this problem, existing approaches [1, 24] focus on demonstrating an upper bound via adversarial evaluation with strong attacks, such as model fine-tuning/pruning, transfer learning and kernels cutoff/supplement. However, existing watermarking techniques are only estimated empirically against attacks that involve model parameter change. Such estimation provide neither any formal guarantee, nor any information against attacks involving perturbation on input images.

To address this gap, we aim to shed light on the susceptibility of several trigger set-based watermarking methods [1, 24] to input-perturbed based attacks. We contend that prevailing methods are insufficient for their purported objective, which is to facilitate model owners in affirming their proprietorship of watermarked DNN models when confronted with input manipulations. We believe that antecedent research has not exhaustively evaluated the robustness of exsiting watermarking algorithms.

Furthermore, we believe that the reliance on empirical evaluations without formal guarantees leaves a gap in the reliability of these methods. In high-stakes scenarios, where the proof of ownership is crucial, the absence of a theoretical framework to support the robustness claims undermines the trustworthiness of watermarking techniques. Thus, there is a pressing need for a methodological shift towards a more rigorous analysis that can withstand the scrutiny of both practical and theoretical assessments.

To bridge this chasm, our work applies formal verification methods to provide formal guarantee against input perturbations that can

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ARTMAN '25, Taipei, Taiwan
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1909-7/25/10
https://doi.org/10.1145/3733821.3763024

be crafted to obscure or remove watermarks without directly modifying the model's parameters. We introduce three major properties to evaluate watermark robustness against various input image attacks: A-robustness, P-robustness, and α-robustness. A-robustness pertains to the adversarial robustness against pixel perturbations in both original images and watermarks. P-robustness concerns the positional resilience of the watermark, ensuring that minor positional shifts do not compromise ownership verification. Lastly, α-robustness relates to the robustness of the alpha channel in alpha blending [15], where slight changes in the alpha value should not affect the ownership verification outcome. By defining and rigorously analyzing A-robustness, P-robustness, and α-robustness, we aim to provide a comprehensive framework that not only withstands input perturbation-based attacks but also lends theoretical credence to the robustness of watermarking methods.

To explore the robustness of existing watermarking methods when facing slight changes on watermarked image, we train watermarked neural models using different watermarking schemes: f_{Adi}, $f_{Content}$, $f_{Unrelated}$ and f_{Noise} on Cifar-10 and MNIST dataset. We adopt state-of-the-art neural network verifier $\alpha - \beta$−crown as our backend verifier to verify the three properties that we defined. Our results demonstrate that existing watermarking techniques lack the ability to resist slight changes on watermarked image.

Contributions: To summarize, our contributions are:

(i) We formally define three novel concepts of robustness that can serve as benchmarks for future watermarking techniques;
(ii) We are the first to apply formal verification methods on watermarking evaluation, providing insight with formal guarantee;
(iii) We provide a comprehensive analysis of the vulnerabilities of current trigger set-based watermarking methods to input perturbation attacks.

2 Preliminaries

2.1 Deep Neural Network Watermarking

A deep neural network comprises essential components, including the input layer that receives initial data, and the hidden layers situated between the input and output layers, where the network performs complex computations. These layers execute different tasks, such as linear or non-linear operations. Fully connected layers within the network manage linear operations, while activation layers oversee non-linear operations, enhancing the network's ability to process and comprehend information efficiently.

In a fully connected layer, each neuron's computation is based on a weighted sum of the outputs from neurons in the preceding layer. Mathematically, this can be expressed as (refer to Fig.1):

$$\forall, 0 \leq i \leq p-1, V[i] = B[i] + \sum_{j=0}^{q-1} W[i][j] \cdot U[j]$$

Here, V represents the output vector of size p for the fully connected layer, B represents the bias vector, U represents the output vector of size q from the preceding layer, and W represents the weight matrix. $W[i][j]$ denotes the weight on the edge between neuron $U[j]$ and neuron $V[i]$. The trainable parameters B and W are updated through backpropagation. In practice and literature, the linear and activation layers are often combined into a single layer.

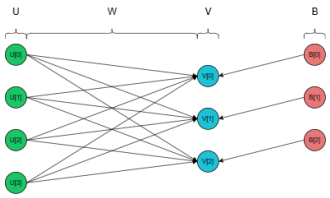


Figure 1: A fully connected layer has 3 neurons (in blue); it's connected to a preceding layer with 4 neurons (in green) and a bias vector (in red). Each edge has an associated weight value.

DNN watermarking [18] refers to the process of embedding a unique watermark into a Deep Neural Network (DNN) model. The purpose of watermarking is to protect intellectual property rights and prevent unauthorized usage or distribution of DNN models.

Mathematically, we can define some notations related to DNN watermarking as follows: Let (f) represent the original DNN model to be watermarked. Let (WI) represent the watermark information to be embedded into the model. Let (f_{WI}) represent the watermarked DNN model, which is obtained by embedding the watermark information (WI) into the original model (f). DNN watermarking process can be divided into two main steps: embedding and extraction.

Embedding: During the embedding process, the watermark (WI) is incorporated into the DNN model (f) to create the watermarked model (f_{WI}). This embedding process can be defined as: [f_W = Embed(f, WI)] Here, (Embed($\cdot$)) denotes the watermark embedding function, which takes the original DNN model (f) and the watermark information (WI) as inputs and produces the watermarked model (f_{WI}).

Extraction: The extraction process involves retrieving the embedded watermark from the watermarked DNN model (f_{WI}). The extraction process can be defined as: [WI' = Extract(f_{WI})] Here, (Extract($\cdot$)) represents the watermark extraction function, which takes the watermarked DNN model (f_{WI}) as input and retrieves the embedded watermark (WI').

Deep neural network (DNN) watermarking techniques can be categorized into two subgroups, based on their respective embedding styles: feature-based and trigger set-based. Feature-based watermarking [2, 14, 18] involves embedding the watermark into the neural network by modifying internal weight parameters. On the other hand, trigger set-based watermarking [1, 5, 7, 9, 24] entails embedding the watermark into the neural network by training the model on a predefined trigger set, in addition to the normal training dataset. This paper primarily aims to provide formal guarantees for trigger set-based neural network watermarking.

2.2 Watermark Removal Attack

A watermark removal attack refers to an unauthorized attempt to remove or modify the embedded watermark in a watermarked DNN model. The objective of this attack is to render the watermark

undetectable or replace it with a different watermark, thereby undermining claims of ownership or tampering detection. The attack aims to generate a surrogate model from the source model, which no longer contains the watermark, while maintaining a comparable level of accuracy on the normal training set.

Current strategies for watermark removal attacks involve techniques such as fine-tuning [10] and parameter pruning [28], which obfuscate the ownership, as well as ownership privacy attacks and ambiguity attacks that aim to reclaim the ownership. These strategies typically involve modifying the model parameters. However, our paper focuses on providing a formal guarantee against attacks that are less costly in practical scenarios, i.e., attacks that do not require extensive modifications to the model parameters.

2.3 Formal Verification in Deep Neural Networks

Neural network verification involves using formal verification to mathematically prove the properties of neural networks. In this methodology, a deep neural network is considered as a mapping function from input to output, and the property description is formalized as a logical formulation containing the inputs and outputs. Different strategies can be employed to verify this property description using the available modeling framework.

Formally, neural network verification can be defined as follows:

Definition 1 (Neural Network Verification)**.** Given a neural network N that maps input vector x to output vector y, and predicates P and Q, verifying N about P and Q is to prove:

$$\forall x, P(x) \Rightarrow Q(N(x)).$$

Equivalently, we need to prove that:

$$P(x) \wedge \neg Q(N(x))$$

is unsatisfiable (UNSAT). When it is satisfiable (SAT), a counterexample x_{adv} will be find such that $P(x_{adv})$ is true and $Q(f(x_{adv}))$ is false.

Bound propagation [16, 19, 20] is a widely adopted technique in neural network verification. For the input layer, the bound is given by the user. As for the hidden layers, the computation varies for linear transformations and non-linear activation functions. For linear transformations, the bound of the neuron can be computed directly using the lower and upper bounds of neurons from the preceding layer. For a transformation of the form:

$$f_{v_i}(z) = \sum_j c_j z_j + b_j$$

The lower and upper bounds are written as:

$$\sum_j \min c_j z_j + b_j \leq f_{v_i}(z) \leq \sum_j \max c_j z_j + b_j.$$

The bound of activation functions can be computed with different methods. One typical way to compute the bound, especially for the monotonic activation functions, is to compute it using the bound for the preceding layer directly. Another possible way is to relax the activation function with two linear functions $f^l_{v_i}$ and $f^u_{v_i}$ such that:

$$\forall x_j \in [z^l_j, z^u_j], f^l_{v_i}(x) \leq f_{v_i}(x) \leq f^u_{v_i}(x)$$

where z^l_j and z^u_j denote the lower and upper bounds for the $j-th$ neuron in the preceding layer. Thus the upper and lower bound of the activation neuron can be given using the upper and lower bound of the two linear relaxations. After the computation of bounds for the hidden layers, the bound of the output neurons can also be computed with the same method.

3 Overview

In this section, we provide an illustrative example for each of the properties introduced in this paper. We consider a scenario where two parties are involved in an attack: a model owner denoted as O, and an adversary denoted as A. On one hand, the model owner O has watermarked their neural network during training using a trigger set. This allows O to demonstrate ownership by obtaining unique model predictions on the trigger set. On the other hand, the adversary A aims to falsify the ownership verification process with the intent of stealing the model from O. Unlike previous research on watermark removal-based attacks for neural networks, A opts for a simpler approach by perturbing the input to attack the ownership verification process without touching the model weights.

3.1 Adversarial Attacks on Watermarked Image (A-Robustness)

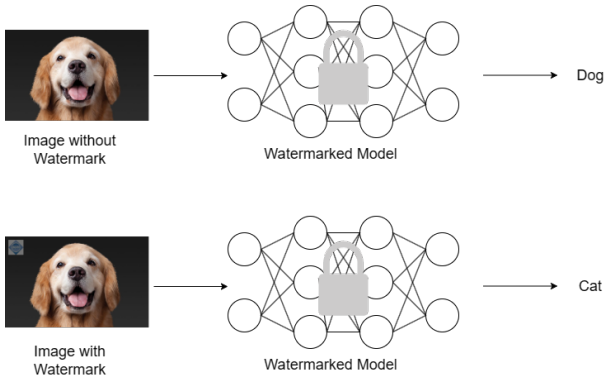


Figure 2: After watermark embedding by O, the watermarked model will classify a dog without watermark as a dog, and classify a dog bearing a watermark as a cat.

As depicted in Figure 2, entity O has employed watermarking techniques during the training process of their model by incorporating a distinct set of triggers that induce specific predictions when applied to the model. In this particular scenario, the trigger is represented by the ACM logo positioned at the top left corner. The watermarked model effectively recognizes an unmodified dog as a dog, but mistakenly classifies a dog with a watermark as a cat. Now during the ownership verification process, instead of investing time and resources into removing the watermark or attacking the model directly, A decides to employ an adversarial attack on input pixels.

As shown in Figure 3, the proposed approach for A involves adding adversarial perturbations to the watermarked image, which is the neural model input. This method is consistent with existing research on adversarial attacks (e.g. PGD [12, 23], FGSM [6, 26]).

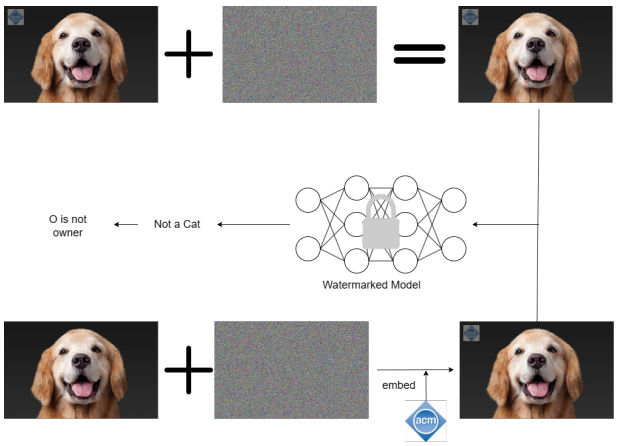


Figure 3: Two ways for *A* to adversarially perturb the watermarked image, thereby deceiving the ownership verification mechanism implemented by *O*.

Consequently, the watermarked deep neural network (DNN) produces outputs that differ from "cat," contrary to the claims made by *O*, while the visual appearance of the watermarked image remains indistinguishable to human observers. However, it is worth considering that during the ownership verification process, *A* may encounter situations where the watermark or trigger set is inaccessible. In such cases, *A* resorts to modifying the pixels of the original image before it is watermarked, as illustrated in the second pathway depicted in Figure 3.

Given these considerations, *O* requires a formal guarantee that any adversarial attack following watermarking consistently fails. If such a guarantee cannot be provided, then the adversarial perturbation applied before watermarking should also fail.

3.2 Positional Attacks on Watermarked Image (P-Robustness)

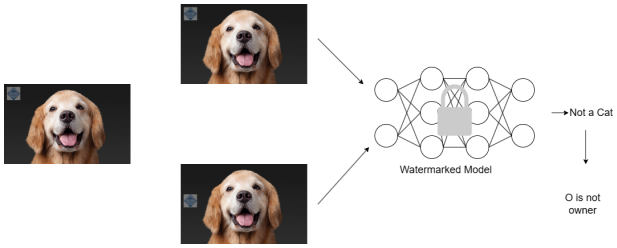


Figure 4: Two ways for *A* to perturb the position of the watermarked image, thereby deceiving the ownership verification mechanism implemented by *O*.

In addition to direct pixel value attacks, *A* has the potential to manipulate the position at which the watermark is embedded during ownership verification. For example, in Fig. 4, the watermark was originally embedded at position (20,20) during training. Initially, *A* may opt to significantly alter the watermark position, such as relocating it to position (200,20) as depicted in Fig. 4. However, such drastic changes would likely be promptly detected by *O*. Consequently, a more strategic approach for *A* would involve making subtle positional adjustments, like shifting the position to (21,21) as illustrated in Fig. 4. Through such positional manipulations, *A* could potentially deceive the ownership verification process.

In light of these considerations, *O* necessitates a formal guarantee that the watermarking technique exhibits global robustness to positional alterations. If achieving global robustness is unfeasible, the scheme should demonstrate resilience to local positional changes. This formal assurance is imperative to safeguard the integrity of the ownership verification procedure in the presence of adversarial attempts to modify the watermark's position.

3.3 Alpha Channel Attacks on Watermarked Image (α-Robustness)

In order to preserve the original content of an image, it is suggested that *O* could employ alpha blending [15] while embedding their watermark into the training image. This technique allows for the watermark to be partially transparent, which has less influence to the original image. However, it should be noted that modifying the value of the alpha channel, as done by *A*, can potentially falsify the ownership claim. It is also anticipated that a watermark with a higher degree of clarity, such as a less transparent watermark, would yield better results compared to a more transparent one.

Figure 5 illustrates the concept of alpha blending watermarking with different alpha values. In this scenario, *O* trains the watermarked model using an alpha value of 100. However, it is expected that the watermarks will still be effective even if *A* slightly alters the alpha value to 101. Additionally, *O* also anticipates that a stronger watermark, with an alpha value of 255, for example, should not fail during ownership verification.

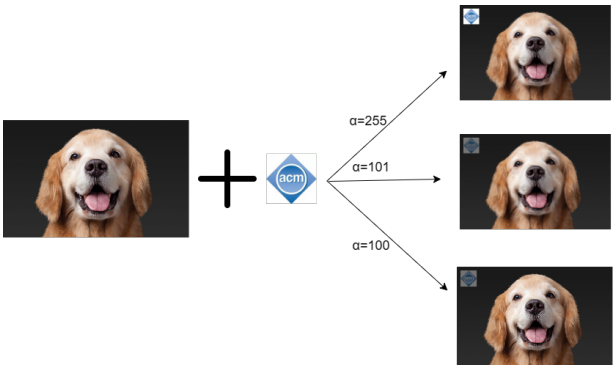


Figure 5: Possible alpha blending watermarking with different alpha value.

4 Methodology

4.1 Trigger Set-Based DNN Watermarking

As shown in Fig. 6, trigger set-based watermarking in neural networks involves two stages, watermark embedding and watermark extraction. In this paper, we do not care about how the owner embed its watermark during training process, but we care about whether

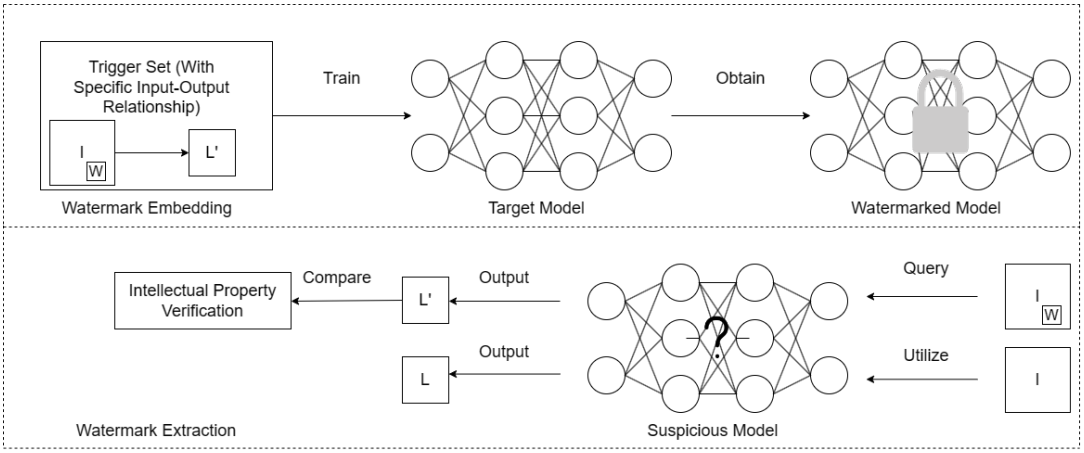


Figure 6: Illustration of Trigger Set-Based Watermarking in Neural Networks, depicting the stages of watermark embedding and extraction.

the embedded watermark can be extracted properly. To this end, we aim to prove that the embedded watermark functions correctly, even when facing attacks on the inputs.

Before delving into the details of our work, we first clarify necessary notions that will be used throughout this paper. Given an input image I whose size is $N \times M$, a well-trained neural network f, and f classify I as $L(I)$. $f_t(I)$ denotes the probability of I being classified to the class t. Besides, let W be the watermark image with size $n \times m$ that is added on the position (p, q), and let α be the value of the alpha channel of the watermark image, which refers to the transparency of the background image. Note that for convenience, we normalize α from $(0, 255)$ to $(0, 1)$ in our paper. Therefore, the watermarked image $I_W(p, q)$ that is used to verify the ownership can be generated via:

$$I_W(p,q)_{i,j} = \begin{cases} I_{i,j} \text{ if } i \notin (p, p+n), j \notin (q, q+m) \\ (W_{i-p,j-q} \times \alpha + I_{i,j} \times (1-\alpha)) \text{ else} \end{cases}$$

, where i, j denote the pixel position. The watermarked image I_W is classified by f as $L(I_W)$. Clearly, a successful ownership verification requires that $L(I_W)$ is correct while maintaining that $L(I)$ is correct. Under this notation, we define below four different types of robustness for the functioning of embedded watermark.

4.2 Problem Statement

Threat Model The adversary $\mathcal{A}$ aims to invalidate ownership verification while preserving model functionality. $\mathcal{A}$ operates under:

- **Knowledge**:
  - No access to watermarking algorithm $W(\cdot)$ or secret key K
  - No white-box knowledge of target model f_θ
- **Capabilities**:
  (1) ***Observability of Visual Features***
  $\mathcal{A}$ can visually/localize the watermark's spatial position (p, q) and estimate transparency α via human inspection or automated analysis (as W's outputs are perceptual objects).
  (2) ***Data Transmission Control***
  $\mathcal{A}$ can manipulate watermarked images $\mathcal{I}_{\text{WM}}$ during their transmission to validation systems (e.g., altering API inputs).
  (3) ***Content-Preserving Operations***
  $\mathcal{A}$ applies ϵ-bounded perturbations: - Spatial shifts: $\delta_p, \delta_q \leq \Delta_{\text{pos}}$ - Transparency changes: $|\delta_\alpha| \leq \Delta_{\text{alpha}}$

A-robustness. Intuitively, any image that is embedded with an watermark should be robust to adversarially perturbed input image. This is two fold. The ownership verification performed by watermarking should be robust to the insignificant changes both on the original image and on the embedded watermark. Thus, we divide it into two sub-types. The first sub-type describes how embedded watermarks should react to the minor changes on the original image before the watermarking process during the extraction period, and guarantees that attackers cannot falsify the ownership by altering the original input image with adversarial perturbations:

Definition 2 (A-Robustness Against Pre-Perturbation)**.** Given a neural network f trained with a watermark W at position (p, q), f is said to be A-robust against pre-perturbation on an image I within a radius ϵ if and only if,

$$||I' - I||_\infty \leq \epsilon \bigwedge L(I'_W(p,q,\alpha)) \neq L(I_W(p,q,\alpha))$$

is unsatisfiable.

The second sub-type of A-robustness describes the robustness of watermark against adversarial attacks after the watermark embedding process during the extraction period:

Definition 3 (A-Robustness Against Post-Perturbation)**.** Given a neural network f trained with a watermark W at position (p, q), f is said to be A-robust against post-perturbation on an image I within a radius ϵ if and only if,

$$||I'_W - I_W||_\infty \leq \epsilon \bigwedge L(I'_W(p,q,\alpha)) \neq L(I_W(p,q,\alpha))$$

is unsatisfiable.

P-robustness. P-robustness refers to positional robustness, that the watermark embedded into an image should be robust to its

positional changes. This requires the watermarked model to recognize the watermark even when it is not at its original position. According to the strength of the concept, we formally define two versions of P-robustness. A weaker version is as follows:

Definition 4 (Local P-Robustness)**.** Given a neural network f trained with a watermark W at position (p, q), f is said to be locally P-robust on an image I within a pair of positional radius (ϵ_p, ϵ_q) if and only if,

$$||p' - p||_\infty \leq \epsilon_p \bigwedge ||q' - q||_\infty \leq \epsilon_q$$
$$\bigwedge L(I_W(p', q')) \neq L(I_W(p, q))$$

is unsatisfiable.

As opposed to local P-robustness, we expect global P-robustness to be a stronger version, that the watermark should be able to be added at any position on the input image. Therefore, we formally define global P-robustness as follows:

Definition 5 (Global P-Robustness)**.** Given a neural network f trained with a watermark W at position (p, q), f is said to be globally P-robust on an image I if and only if,

$$0 \leq p \leq N - n \bigwedge 0 \leq q \leq M - m$$
$$\bigwedge L(I_W(p', q', \alpha)) \neq L(I_W(p, q, \alpha))$$

is unsatisfiable.

α-robustness. Different to two previous defined robustness, α-robustness is related to the alpha channel, which is commonly used when overlapping two different image. According to the computation on the alpha channel, the larger α is, the less transparent the watermark is. Intuitively, when a neural network is trained with a trigger set-based watermark of a certain *alpha*, we have two expectations. On the one hand, it is expected that the watermark can still claim the ownership even if α is insignificantly perturbed. On the other hand, the watermarked image should still function even if its alpha channel has a value smaller than α, which indicates that the watermark is less transparent.

Accordingly, we formally defined a local version of α-robustness and a global version of α-robustness.

Definition 6 (Local α-Robustness)**.** Given a neural network f trained with a watermark W at position (p, q), f is said to be locally α-robust on an image I within a radius on alpha channel ϵ_α if and only if,

$$||\alpha' - \alpha||_\infty \leq \epsilon_\alpha \bigwedge L(I_W(p, q, \alpha')) \neq L(I_W(p, q, \alpha))$$

is unsatisfiable.

As opposed to local α-robustness, we expect global α-robustness to be a stronger version, that the watermark should be able to be added with any *alpha* larger than α during training. Therefore, we formally define global α-robustness as follows:

Definition 7 (Global α-Robustness)**.** Given a neural network f trained with a watermark W at position (p, q), f is said to be globally α-robust on an image I if and only if,

$$\alpha' \geq \alpha \bigwedge L(I_W(p, q, \alpha')) \neq L(I_W(p, q, \alpha))$$

is unsatisfiable.

Algorithm 1: A-robustness Verification using Backend Verifier V on Neural Network f

```
Define verify (f, Q, P):
Input: Neural network f, output property Q, input property P
Output: SAT or UNSAT
if verify (f, P = B_∞(I_W, ε), Q) is UNSAT then
    return UNSAT for A-robustness verification against both post-perturbation and per-perturbation
else
    return SAT for A-robustness verification against post-perturbation
    if verify (f, P = B_∞(I_W, ε) ⋃ B_∞(W, 0), Q) is UNSAT then
        return UNSAT for A-robustness verification against pre-perturbation
    else
        return SAT for A-robustness verification against pre-perturbation
```

4.3 Verification Problem Transform

Given these three properties and their corresponding verification problems, it is necessary to furnish a formal guarantee for the watermarked model that ensures non-violation of these properties. This will impede A from easily falsifying ownership verification.

A-Robustness. A-robustness, similar to adversarial robustness in previous studies, presents a straightforward verification problem. It is important to note that if the neural network f is A-robust against post-perturbation, it will also be A-robust against pre-perturbation. This is because a pre-perturbed watermarked image is always a special case within the set of post-perturbed watermarked images. Furthermore, from the perspective of the owner O, it is only natural for O to expect that the watermarked neural network remains impervious to even the strongest attacks from A. Hence, the verification procedure for A-robustness is designed to prioritize the verification of post-perturbation robustness. If the model returns SAT, only then will it be subjected to verification against pre-perturbation.

The backend neural network verifier V, as shown in Algorithm 1, takes as input a neural network f, an output property Q, and an input property P that can be represented as an L_∞-ball, $B_\infty(x, \epsilon)$, or a union of several L_∞-balls, $\bigcup B\infty(x_i, \epsilon_i)$. It outputs the verification result SAT or UNSAT. UNSAT means that the property is guaranteed, while SAT means that there exist counterexamples. O should first use V to verify A-robustness against post-perturbation, which is $\textsc{Verify}(f, B_\infty(I_W, \epsilon), Q)$. If the result is UNSAT, then there is no need to verify A-robustness against pre-perturbation since it will be automatically satisfied. Otherwise, O should first use V to verify A-robustness against pre-perturbation, which is $\textsc{Verify}(f, B_\infty(I_W, \epsilon) \bigcup B_\infty(W, 0), Q)$.

P-Robustness. The problem of verifying P-robustness has two important differences compared to verifying A-robustness. Firstly, the position (p, q) is not a direct input for a neural network. Therefore, we can't use our backend verifier to confirm P-robustness. However, the second important characteristic is that the positional

changes are discrete, which means they can be verified case by case. Consequently, we opt to create all the images with potential changes in the watermark position and compute the output using the watermarked neural network f. Additionally, global P-robustness is manifestly a more potent form of local P-robustness. Unlike A-robustness, the verification process of local P-robustness is essentially part of the verification process of global P-robustness.

In Algorithm 2, we run through all the pixels (i, j) in the original image and generate all possible watermarked images with the watermark embedded at (i, j) without exceeding the original image. Then we define a function $dist((i, j), (p, q)) = max(|i-p|, |j-q|)$ to calculate the pixels' distance. We initially set the distance d as 1 and check all instances with the watermark at (i, j) and $dist((i, j), (p, q)) = d$ to see if they violate P-robustness. Subsequently, we gradually increase the value of d and stop when there are no more watermarked images to be verified, or if the check function returns SAT for some instances. When the check function returns SAT for an instance with distance d to position (p, q), this automatically signifies the failure of P-robustness verification for those larger than d.

Algorithm 2: P-robustness Verification on Neural Network f

```
Define dist ((i, j), (p, q)) = max(|i − p|, |j − q|)
Define ImageSet=∅
for each each pixel (i, j) in I do
    generate I_W(i, j) and store it into ImageSet if possible
d = 0
while ImageSet≠ ∅ and d ≤ ε_p do
    check all instance I_W(i, j) with dist ((i, j), (p, q)) = d
    Delete checked instances from ImageSet
    if check return SAT then
        return SAT for dist ((i, j), (p, q)) ≥ d
    else
        set UNSAT for dist ((i, j), (p, q)) ≤ d
    d+=1
```

α-Robustness. Similar to P-robustness, α-robustness cannot be verified directly using the backend verifier V that was used for A-robustness. However, it shares the same continuous data characteristic as A-robustness, making it impossible to test all instances with a perturbed alpha. Therefore, we propose a network transformation strategy for the watermarked network f, generating a new network f_α that takes α as input and outputs the prediction for $I_W(p, q, \alpha)$. We observe that the generation of the watermarked image can be rewritten as:

$$I_W(p,q)_{i,j} = \begin{cases} I_{i,j} \text{ if } i \notin (p, p+n),\ j \notin (q, q+m) \\ (W_{i-p,j-q} - I_{i,j}) \times \alpha + I_{i,j} \text{ else} \end{cases}$$

Therefore, we construct a new network layer with one input neuron and $N \times M$ output neurons. We assign the bias b_{ij} as $b_{i,j} = I_{i,j}$, and we assign the weights w_{ij} as follows:

$$w_{i,j} = \begin{cases} 0 \text{ if } i \notin (p, p+n),\ j \notin (q, q+m) \\ (W_{i-p,j-q} - I_{i,j}) \text{ else} \end{cases}$$

We then concatenate this layer with f to obtain f_α, which takes α as input and outputs the same prediction as $f(I_W)$.

Algorithm 3: α-robustness Verification using Backend Verifier V on Neural Network f

```
Use transform to construct f_α that takes α as input and
  outputs the same prediction as f(I_W):
verify (f_α, P = [α − ε_α, α + ε_α], Q)
verify (f_α, P = [α, 1], Q)
```

The overall verification is shown in Algorithm 3. Once the modified network f_α is constructed, the algorithm performs two verification steps using the backend verifier V and verifies the local $\alpha-$robustness and the global α-robustness separately.

Backend Verifier Selection We employ alpha-beta-Crown [20, 24] as our backend verifier due to its efficiency in handling neural network verification problems. This tool operates by combining linear relaxation with branch-and-bound techniques to compute tight output bounds for DNNs under input constraints. While alpha-beta-Crown is not formally complete (as with most practical verifiers), it provides sound guarantees – any robustness property it certifies holds definitively. The verifier's strength lies in its scalability to medium-sized networks, achieved through optimizations like GPU acceleration and parallel processing. For our watermark robustness verification, we leverage its ability to handle activation functions common in image classification models and its support for multi-neuron constraints. Though the base version has inherent limitations in completeness, recent extensions like beta-Crown have shown improved verification coverage, which we implement where computationally feasible given our evaluation scale.

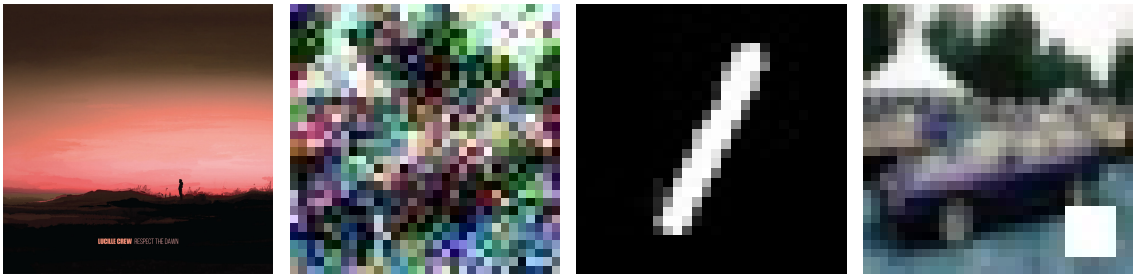
5 Experiments

Within this section, we delineate the outcomes of our conducted experiments. Our elaboration encompass the configuration of our experimentation, the methods of watermarking employed, and the architectural nuances of the models. Subsequently, we enumerate the findings obtained from our verification process for each specific robustness attribute.

5.1 Experimental Setup

Our experiment is conducted on a Linux server running Ubuntu 18.04, equipped with an Intel Xeon E5-2678 v3 processor and a GPU of Nvidia RTX 4090. We implement all the chosen watermark schemes with PyTorch running as their backend. We adopt state-of-the-art verification tool alpha-beta-Crown [20, 24] as our backend verifier, as mentioned in previous sections.

Watermarking Schemes. The watermarking schemes evaluated in our study follow a common embedding and extracting strategy, which has been widely adopted in previous research. The embedding process entails fine-tuning the model using the watermarking key, while extracting a message from a surrogate model is achieved by requesting labels for the key images. The primary difference lies in the selection of the watermarking keys. We have selected four watermarking schemes, which are illustrated in Fig. 7 and described below. Adi et al. [1] utilize a confidential watermarking



(a) Adi [1] (b) Noise [25] (c) Unrelated [25] (d) Content [25]

Figure 7: Samples of the watermarked imgae used in the four chose watermarking schemes

key comprising abstract, out-of-distribution images. An image label is randomly selected from all classes, excluding the image's actual label. We note the watermarked model using this scheme as f_{Adi}. Zhang et al. [25] proposed three distinct schemes that label key images with a randomly selected class, denoted as $f_{Content}$, f_{Noise}, and $f_{Unrelated}$.

These schemes are described as follows: (1) Content: In this scheme, the secret watermarking keys are images sampled from a single source class, which are then perturbed by applying a secret, additive mask (e.g., a white square covering part of the image for both CIFAR-10 and MNIST). (2) Noise: This scheme involves utilizing a secret, additive mask composed of random Gaussian noise. (3) Unrelated: The watermarking keys in this scheme are images sampled from a different domain that is unrelated to the source model's domain. We choose figures with "m" from EMNIST dataset for MNIST and figures with "1" from MNIST dataset for CIFAR-10.

We watermarked the DNN models trained on the two datasets using the previously mentioned watermarking schemes, yielding a total of 16 models. It is worth noting that by our definition, f_{Adi}, $f_{Unrelated}$ and f_{Noise} will only be considered under A-robustness against post-perturbation.

Datasets, Model Architecture and Metrics. We embed watermarks into source models trained on the image classification datasets MNIST and CIFAR-10. For the MNIST dataset, our evaluation is based on two architectures: **MLP-9-100** and **CNN-Small** from [21, 22]. For the CIFAR10 dataset, we also investigate two architectures: **CIFAR-CNN-A** and **ResNet-2b** from [20].

During evaluation, we test each predefined robustness property (A-robustness, P-robustness, α-robustness) against 100 watermarked trigger images. An instance is labeled SAT (unsafe) when our verifier confirms that the watermarked model fails to maintain correct watermark detection under the specified adversarial condition for that image. The reported unsafe instances therefore represent the count of trigger images vulnerable to each attack type.

5.2 A-Robustness of Watermarking Schemes

In this section, we analyze the A-robustness of each watermarking scheme against pre-perturbation and post-perturbation. The goal of this analysis is to prove that whether any watermarking scheme can be considered robust to minor perturbation on pixels.

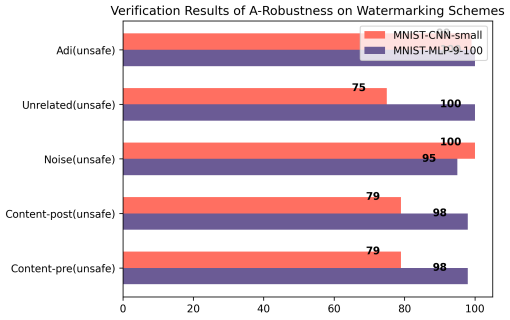


Figure 8: Experimental results on verifying A-robustness on MNIST, showing the number of SAT (unsafe) instances for each watermarking scheme.

As illustrated in Figure 8, the majority of watermarking schemes exhibit SAT (satisfactory) results on the MNIST dataset after perturbation. Therefore, we can make a more obvious comparison based on their SAT instances. For the CNN-small model, the Adi watermarking scheme demonstrates 99 instances that are deemed unsafe, indicating potential weaknesses in its ability to protect watermarked data. The Unrelated scheme shows 75 unsafe instances, suggesting vulnerabilities in its resistance to potential attacks. The Noise scheme has the highest number of unsafe instances at 100, indicating significant vulnerabilities in maintaining the integrity of the watermarked content. Both the Content-post and Content-pre schemes have 79 unsafe instances, indicating comparable vulnerability in terms of A-Robustness. For the MLP-9-100 model, both the Adi and Unrelated schemes exhibit 100 unsafe instances, suggesting that they may not effectively protect watermarked content against various attacks. The Noise scheme performs relatively better with 95 unsafe instances, indicating improved performance compared to the previous two schemes. Both the Content-post and Content-pre schemes have 98 unsafe instances, showing similar levels of vulnerability in terms of A-Robustness. This implies that an adversary (A) can easily falsify the ownership claimed by the original owner (O) through simple alterations of pixel values.

The verification results for watermarking schemes on the CIFAR-10 dataset are presented in Fig. 9. Obviously, the analysis indicates that the Adi watermarking scheme has the highest number of unsafe

instances for both datasets, with 88 instances for the CIFAR10-CNN dataset and 91 for the CIFAR10-resnet2b dataset. This suggests that the Adi scheme may not be the most effective for these particular datasets, as its labels are randomly selected. Moreover, the Unrelated watermarking scheme has the second-highest number of unsafe instances. This finding suggests the similarity between the Unrelated scheme and the Adi scheme. Additionally, the Noise watermarking scheme has a relatively low number of unsafe instances for both datasets, indicating its potential effectiveness for these datasets compared to other schemes. The Content-post and Content-pre watermarking schemes demonstrated the same number of unsafe instances for both datasets, suggesting a similar level of effectiveness for these watermarking schemes.

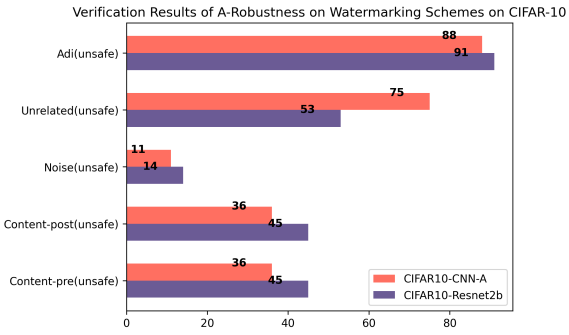


Figure 9: Experimental results on verifying A-robustness on CIFAR-10, showing the number of SAT (unsafe) instances for each watermarking scheme.

The fundamental fragility across schemes stems from how watermarks are embedded without robustness constraints. During standard training, models learn to detect watermarks on clean samples but don't develop resistance to perturbed variants. This creates decision boundaries where minor pixel changes - well within our verification bounds - can flip predictions.

The dramatic performance gap between MNIST and CIFAR-10 (e.g., Noise scheme fails completely on MNIST but succeeds on CIFAR-10) results from dataset characteristics rather than methodology flaws: (1) *Feature sensitivity*: MNIST's high-contrast, simple features make watermarks dominant yet easily disrupted patterns, while CIFAR's complex textures naturally absorb perturbations; (2) *Model capacity*: Larger CIFAR models (ResNet) develop more distributed feature representations where watermarks aren't single points of failure; (3) *Data diversity*: CIFAR's natural variations provide implicit regularization against watermark overfitting. Thus, robustness depends critically on how watermark patterns interact with dataset-specific features and model architectures.

Regarding A-robustness against pre-perturbation, experiments were only conducted with the Content scheme. However, the results indicate that it is exactly the same as post-perturbation. This finding suggests that during watermarking, the watermark itself may not be the sole factor influencing the model's predictions. It is more likely that the prediction is made considering both the watermark and the unwatermarked portion of the image.

5.3 P-Robustness of Watermarking Schemes

Similarly, P-robustness is a concept that specifically applies to the $f_{Content}$ watermarking scheme. In the context of other watermarking schemes, the positioning of the watermark either encompasses the entire image or lacks a well-defined position. Consequently, our experimentation focuses on four neural network models trained using the "Content" scheme. The specific content embedded in the figures consists of 6×6 white squares, as described in [25].

In Fig. 10, we present the results of verifying the P-robustness of the watermark in images across the four neural network models. This experiment assesses the resilience of the watermark to positional changes, considering both local and global positional radii. The local versions encompass varying levels of positional radii, denoted as "local-1," "local-2," and "local-3," corresponding to different degrees of local P-robustness with positional radii 1, 2, and 3, respectively. The term "Global" signifies global P-robustness.

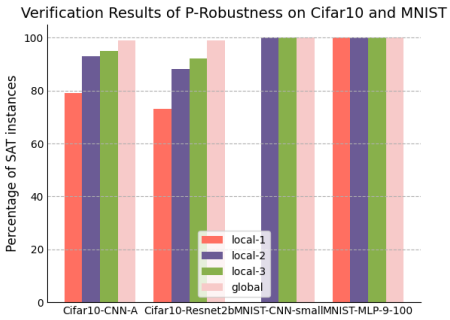


Figure 10: Experimental results on verifying P-robustness on the selected models and datasets under "Content" watermarking scheme, showing the number of SAT (unsafe) instances for each combination of models and datasets.

The results reveal susceptibility of the watermarking scheme to positional changes of the watermark. In the case of the CIFAR models (Cifar10-CNN-A and Cifar10-Resnet2b), all levels of local P-robustness exhibit a significant proportion of unsafe outcomes, indicating their vulnerability to positional alterations. Furthermore, the number of safe instances decreases substantially as the positional radii increase. Global P-robustness also shows a decrease in safe instances, with only 1% of outcomes considered safe.

On the other hand, when considering the MNIST models (MNIST-CNN-small and MNIST-MLP-9-100), we observe that the CNN-small model trained on the MNIST dataset showcases remarkable P-robustness at a local radius of 1. This can be attributed to both the simplistic characteristics of the MNIST dataset and the spatial feature extraction capability of CNN models. However, there are no instances deemed safe for any level of P-robustness with radii greater than 1. Additionally, the MLP model trained on MNIST fails to provide formal guarantees for any positional changes.

5.4 α-Robustness of Watermarking Schemes

In this section, we undertake an analysis of the α-robustness exhibited by each of the watermarking schemes. For this purpose,

we perform experiments on CIFAR-10, using solely $f_{Content}$ based schemes. This decision is motivated by two primary factors: firstly, alpha blending can only be precisely defined within the context of $f_{Content}$; secondly, MNIST, being a dataset with only one channel, is not particularly well-suited for the implementation and evaluation of alpha blending techniques.

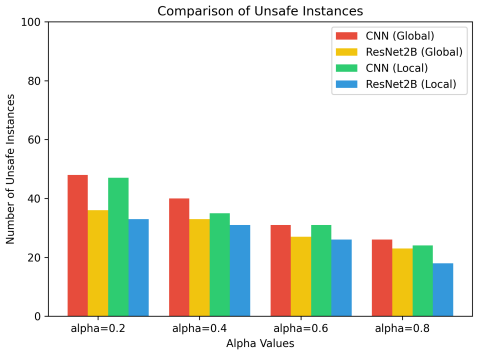


Figure 11: Experimental results on verifying α-robustness on CIFAR-10 with CNN-small and Resnet2b under "Content" watermarking scheme, showing the number of SAT (unsafe) instances for different α value.

Based on Fig. 11, several trends and comparisons can be observed. Firstly, it is evident that the number of unsafe instances generally decreases as the alpha value increases for all scenarios. This observation suggests that increasing the alpha blending parameter leads to improved performance in terms of reducing the number of unsafe instances. Additionally, it is worth noting that for an alpha value of 0.2, the number of unsafe instances is highest across all scenarios compared to other alpha values. This finding implies that the lowest alpha blending level presents the most challenging case for training a robust model, demanding careful consideration. This is reasonable since low alpha blending generally implies that the watermarked image carries limited information about the watermark. Finally, as the alpha value increases, the disparity in the number of unsafe instances between scenarios tends to diminish. This trend may suggest that increasing the alpha value has a more significant impact on enhancing the α-robustness of models, irrespective of the architecture or training method employed.

6 Related Works

Recent deployments of watermarking in large models [4, 27] confirm the significance of input-level threats for language models. However, constrained by the scalability limitations of current neural network verification tools, this work focuses exclusively on computer vision tasks. Our study specifically addresses input perturbation attacks targeting trigger images during ownership verification. This threat differs fundamentally from model modification attacks (e.g., fine-tuning [1] or model extraction [9]), which require access to model parameters. Input perturbation attacks remain feasible when adversaries intercept verification queries—a realistic risk in API-based model services.

Robustness testing has become essential for evaluating AI systems. Research in deep neural networks often manifests as adversarial attack studies [3, 6, 13, 17]. Within watermarking specifically, the goal is to generate test cases that empirically reveal robustness limitations. A comprehensive analysis revealed that none of the surveyed watermarking schemes demonstrated robustness in practice when systematically evaluated against diverse attack scenarios [11]. Furthermore, it is indicated via empirical evaluation that state-of-the-art trigger-based watermarks fail to reliably prove ownership due to two common flaws [8]: incomplete adversarial evaluation and neglected adaptive attacks. Notably, 11 representative schemes exhibit vulnerabilities to at least two non-adaptive attacks. These findings challenge the fundamental assumptions behind many watermarking approaches and calls for more rigorous evaluation methodologies.

Despite their empirical value, testing techniques cannot provide worst-case guarantees. This limitation motivates our adoption of formal verification methods. We employ alpha-beta-Crown as our verification backbone, which builds on the CROWN framework [24]. While CROWN advanced scalability through linear function-based neuron bounding, it sacrifices completeness. To address this, we integrate Branch-and-Bound (BaB) with GPU acceleration following [20], achieving both completeness and scalability.

7 Conclusion

We tackle the issue of robustness in deep neural network (DNN) watermarking techniques in this study. Although many watermarking schemes based on trigger sets claim to be robust, their validation often only involves attacks that manipulate network weights to remove the watermark. This limited evaluation and lack of formal guarantees for perturbed input-based attacks give rise to uncertainties regarding the practical application and reliability of these schemes. To address this, we have formally defined three types of input-related robustness—A-robustness, P-robustness, and α-robustness—for trigger set-based watermarking schemes. We conducted a comprehensive evaluation of four representative watermarking schemes using the CIFAR-10 and MNIST image classification datasets, focusing on formal verification. The results indicate that none of the surveyed watermarking schemes demonstrate practical robustness, highlighting the neglect of simple input-based attacks during the training process of these schemes.

Our future work encompasses several areas. Firstly, we will explore more efficient verification tools for watermarking in this context. Secondly, we will apply formal verification methods to existing watermark removal attacks, such as fine-tuning and weight pruning. Finally, we intend to utilize the verification results to guide the watermarking process, aiming to achieve more secure and robust watermarks.

Acknowledgment

This work is supported by the NSFC-ISF Joint Program with Grant No. 62161146001.

References

[1] Yossi Adi, Carsten Baum, Moustapha Cissé, Benny Pinkas, and Joseph Keshet. 2018. Turning Your Weakness Into a Strength: Watermarking Deep Neural

Networks by Backdooring. In *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*. USENIX Association, 1615–1631.

[2] Huili Chen, Bita Darvish Rouhani, Cheng Fu, Jishen Zhao, and Farinaz Koushanfar. 2019. DeepMarks: A Secure Fingerprinting Framework for Digital Rights Management of Deep Learning Models. In *Proceedings of the 2019 on International Conference on Multimedia Retrieval, ICMR 2019, Ottawa, ON, Canada, June 10-13, 2019*. ACM, 105–113.

[3] S. Chen, Z. He, C. Sun, J. Yang, and X. Huang. 2022. Universal Adversarial Attack on Attention and the Resulting Dataset DAmageNet. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 4 (2022), 2188–2197.

[4] Sumanth Dathathri, Abigail See, Sumedh Ghaisas, Po-Sen Huang, Rob McAdam, Johannes Welbl, Vandana Bachani, Alex Kaskasoli, Robert Stanforth, Tatiana Matejovicova, Jamie Hayes, Nidhi Vyas, Majd Al Merey, Jonah Brown-Cohen, Rudy Bunel, Borja Balle, A. Taylan Cemgil, Zahra Ahmed, Kitty Stacpoole, Ilia Shumailov, Ciprian Baetu, Sven Gowal, Demis Hassabis, and Pushmeet Kohli. 2024. Scalable watermarking for identifying large language model outputs. *Nat.* 634, 8035 (2024), 818–823.

[5] Lixin Fan, Kam Woh Ng, and Chee Seng Chan. 2019. Rethinking Deep Neural Network Ownership Verification: Embedding Passports to Defeat Ambiguity Attacks. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. 4716–4725.

[6] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and Harnessing Adversarial Examples. In *3rd International Conference on Learning Representations*. San Diego, CA, USA, 11 pages.

[7] Hengrui Jia, Christopher A. Choquette-Choo, Varun Chandrasekaran, and Nicolas Papernot. 2021. Entangled Watermarks as a Defense against Model Extraction. In *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*. USENIX Association, 1937–1954.

[8] Suyoung Lee, Wonho Song, Suman Jana, Meeyoung Cha, and Sooel Son. 2023. Evaluating the Robustness of Trigger Set-Based Watermarks Embedded in Deep Neural Networks. *IEEE Trans. Dependable Secur. Comput.* 20, 4 (2023), 3434–3448.

[9] Zheng Li, Chengyu Hu, Yang Zhang, and Shanqing Guo. 2019. How to prove your model belongs to you: a blind-watermark based framework to protect intellectual property of DNN. In *Proceedings of the 35th Annual Computer Security Applications Conference, ACSAC 2019, San Juan, PR, USA, December 09-13, 2019*. ACM, 126–137.

[10] Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2018. Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks. In *Research in Attacks, Intrusions, and Defenses - 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10-12, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 11050)*. Springer, 273–294.

[11] Nils Lukas, Edward Jiang, Xinda Li, and Florian Kerschbaum. 2022. SoK: How Robust is Image Classification Deep Neural Network Watermarking?. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*. IEEE, 787–804.

[12] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *6th International Conference on Learning Representations, (ICLR 2018)*.

[13] S. M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard. 2016. DeepFool: A Simple and Accurate Method to Fool Deep Neural Networks. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), 2574–2582.

[14] Bita Darvish Rouhani, Huili Chen, and Farinaz Koushanfar. 2019. DeepSigns: An End-to-End Watermarking Framework for Ownership Protection of Deep Neural Networks. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2019, Providence, RI, USA, April 13-17, 2019*. ACM, 485–497.

[15] Bo Shen, Ishwar K. Sethi, and Vasudev Bhaskaran. 1998. DCT Domain Alpha Blending. In *Proceedings of the 1998 IEEE International Conference on Image Processing, ICIP-98, Chicago, Illinois, USA, October 4-7, 1998*. IEEE Computer Society, 857–861.

[16] S.Wang and K.Pei and J.Whitehouse and J.Yang and S.Jana. 2018. Efficient Formal Safety Analysis of Neural Networks. In *31st Annual Conference on Neural Information Processing Systems, (NeurIPS 2018)*. 6369–6379.

[17] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. 2014. Intriguing properties of neural networks. In *2nd International Conference on Learning Representations, (ICLR 2014)*.

[18] Yusuke Uchida, Yuki Nagai, Shigeyuki Sakazawa, and Shin'ichi Satoh. 2017. Embedding Watermarks into Deep Neural Networks. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval, ICMR 2017, Bucharest, Romania, June 6-9, 2017*. ACM, 269–277.

[19] S. Wang, K. Pei, J. Whitehouse, J. Yang, and S. Jana. 2018. Formal Security Analysis of Neural Networks using Symbolic Intervals. In *27th USENIX Security Symposium, (USENIX Security 2018)*. 1599–1614.

[20] Shiqi Wang, Huan Zhang, Kaidi Xu, Xue Lin, Suman Sekhar Jana, Cho-Jui Hsieh, and J. Zico Kolter. 2021. Beta-CROWN: Efficient Bound Propagation with Per-neuron Split Constraints for Neural Network Robustness Verification. In *34th Annual Conference on Neural Information Processing Systems, (NeurIPS 2021)*. 29909–29921.

[21] Eric Wong and Zico Kolter. 2018. Provable defenses against adversarial examples via the convex outer adversarial polytope. In *International Conference on Machine Learning*. PMLR, 5286–5295.

[22] Eric Wong, Frank Schmidt, Jan Hendrik Metzen, and J Zico Kolter. 2018. Scaling provable adversarial defenses. *Advances in Neural Information Processing Systems* 31 (2018).

[23] Haotian Xue, Alexandre Araujo, Bin Hu, and Yongxin Chen. 2023. Diffusion-Based Adversarial Sample Generation for Improved Stealthiness and Controllability. In *NeurIPS 2023*.

[24] Huan Zhang, Tsui-Wei Weng, Pin-Yu Chen, Cho-Jui Hsieh, and Luca Daniel. 2018. Efficient Neural Network Robustness Certification with General Activation Functions. In *31st Annual Conference on Neural Information Processing Systems 2018, (NeurIPS 2018)*. 4944–4953.

[25] Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph. Stoecklin, Heqing Huang, and Ian M. Molloy. 2018. Protecting Intellectual Property of Deep Neural Networks with Watermarking. In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security, AsiaCCS 2018, Incheon, Republic of Korea, June 04-08, 2018*. ACM, 159–172.

[26] Linjun Zhang, Zhun Deng, Kenji Kawaguchi, Amirata Ghorbani, and James Zou. 2021. How Does Mixup Help With Robustness and Generalization?. In *ICLR 2021*.

[27] Chaoyi Zhu, Jeroen Galjaard, Pin-Yu Chen, and Lydia Y. Chen. 2024. Duwak: Dual Watermarks in Large Language Models. In *Findings of the Association for Computational Linguistics, ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). 11416–11436.

[28] Michael Zhu and Suyog Gupta. 2018. To Prune, or Not to Prune: Exploring the Efficacy of Pruning for Model Compression. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Workshop Track Proceedings*. OpenReview.net.